

PixelScore

Sistema de Evaluación Automática de Calidad de Imágenes para E-commerce

Informe Técnico — Entrega Final · Procesamiento de Imágenes
Jefferson Gutierrez Tovar · Juan Joya · Mayo 2026

PixelScore analiza fotografías de productos mediante cinco filtros de visión computacional (nitidez, iluminación, contraste, uniformidad del fondo y exposición) y las clasifica en tres categorías: **Profesional, Aceptable o Deficiente**. La decisión se refuerza con una CNN entrenada sobre 6.000 imágenes que alcanzó **99,50 % de precisión de validación**. El resultado se entrega en una interfaz web con score global, métricas individuales, recomendaciones y visualización de filtros.

1. Solución y Valor Real · 40 %

Problema real identificado

Los vendedores de e-commerce (MercadoLibre, Shopify, Instagram Shop) publican productos con fotos de baja calidad: mal iluminadas, borrosas o con fondos sucios. Hoy esa revisión se hace **a mano**, sin criterios objetivos, generando rechazos tardíos y pérdida de ventas. El **75 % de los compradores en línea basa su decisión exclusivamente en la fotografía**. No existe una herramienta accesible que evalúe la foto antes de publicarla.

Contexto y usuario objetivo

El usuario es el **vendedor independiente o PYME** que gestiona su catálogo desde el celular o portátil. El momento de uso es justo **antes de subir la foto**: fotografía el producto, abre PixelScore, arrastra la imagen y recibe la evaluación en menos de 2 segundos. Condiciones: luz variable, fondos improvisados, cámara de gama media.

Impacto y justificación

Beneficio	Descripción
Ahorro de tiempo	El sistema evalúa en < 2 s. Una persona tarda 2–5 min en decidir si una foto es publicable.
Reducción de errores	Criterios matemáticos objetivos e iguales para todas las fotos, sin subjetividad.
Mejora de ventas	Fotos de mayor calidad aumentan la tasa de conversión entre 20–40 % en e-commerce.
Retroalimentación	El vendedor recibe recomendaciones específicas, no solo un rechazo genérico.

Demo del producto

El sistema opera de forma autónoma sin intervención manual:

1. El usuario sube imagen JPG/PNG/WEBP desde la interfaz web.
2. Flask recibe la imagen y lanza el ejecutable C++ compilado (build/evaluador).
3. El ejecutable calcula 5 métricas con OpenCV y consulta el modelo CNN (.onnx).
4. Python genera 7 filtros visuales y los devuelve como imágenes base64.
5. El navegador muestra score, categoría, recomendaciones y filtros en < 2 s.
6. La imagen temporal se elimina automáticamente del servidor.

2. Procesamiento de Imágenes · 30 %

Técnicas aplicadas y justificación

Cinco métricas independientes sobre imagen en escala de grises (BGR→GRAY):

Nitidez — Varianza del Laplaciano

Detecta bordes en todas las direcciones. Alta varianza = imagen nítida. Preferido sobre Sobel/Canny por su sensibilidad omnidireccional. Se añade penalización cuadrática por brillo bajo para que el ruido nocturno no infle falsamente el score.

Iluminación — Luminancia media (canal V en HSV)

Canal V mide el brillo percibido. Rango ideal: 90–170 sobre 255. HSV elegido frente a LAB por su interpretabilidad directa sin transformaciones complejas.

Contraste — Desviación estándar de píxeles

Alta desviación = alto contraste visual. Más robusto que rango dinámico (max-min) ante píxeles atípicos.

Uniformidad del fondo — Desv. estándar en franja periférica

Analiza el borde del encuadre donde suele estar el fondo. Baja varianza = fondo liso. Más rápido que segmentación GrabCut sin perder información útil.

Exposición — Fracción de píxeles saturados

Cuenta píxeles blancos (>250) y negros (<10). Si superan el 5 %, hay zonas quemadas u oscuras inaceptables.

Pipeline coherente

Etapa	Componente	Detalle
1. Adquisición	Flask (Python)	Imagen recibida vía POST multipart desde el navegador.
2. Preprocesamiento	OpenCV (C++)	BGR → escala de grises → redimensionado (480px).
3. Extracción	5 funciones C++	Cada métrica produce valor numérico + score 0–100.
4. Clasificación IA	CNN (.onnx)	Normalización 0–1 → OpenCV DNN → softmax → categoría.
5. Visualización	OpenCV (Python)	7 filtros como imágenes base64 para el frontend.
6. Salida	HTML + JavaScript	Score circular, barras, recomendaciones, grilla de filtros.

Resultados en condiciones reales

Caso	Imagen	Condición	Score	Resultado
1	Reloj sobre fondo blanco	Ideal: buena luz, fondo limpio	~77	✓ Profesional
2	Audífonos sobre mesa de madera	Fondo no uniforme, luz aceptable	~62	✓ Aceptable
3	Bicicleta en movimiento, noche	Oscuro, borroso, fondo ruidoso	~30	✓ Deficiente

Caso de fallo documentado: el Laplaciano asignaba score máximo de nitidez a imágenes oscuras y ruidosas. Se corrigió con penalización cuadrática: $lapVar \times (brillo/120)^2$ cuando el brillo medio es menor a 120, eliminando el falso positivo de nitidez.

3. Uso de Inteligencia Artificial · 30 %

Modelo o enfoque elegido — Por qué CNN

Alternativa evaluada	Razón del descarte
SVM / Random Forest	No captura patrones espaciales; requiere ingeniería manual de características.
ResNet / EfficientNet preentrenado	Demasiado pesado para CPU; latencia > 5 s; innecesario para 3 clases simples.
YOLO / detección de objetos	Orientado a localización, no a calidad global de imagen.
CNN propia ligera ← ELEGIDA ✓	Arquitectura ajustada: 3 conv + FC, entrada 128×128, inferencia < 200 ms en CPU.

Arquitectura de la CNN

Capa	Configuración	Propósito
Conv2D + ReLU	32 filtros 3×3, padding=1	Detecta bordes y texturas básicas
MaxPool2D	2×2	Reduce dimensionalidad a la mitad
Conv2D + ReLU	64 filtros 3×3, padding=1	Patrones de mediana complejidad
MaxPool2D	2×2	Segunda reducción espacial
Conv2D + ReLU	128 filtros 3×3, padding=1	Patrones de alta complejidad
MaxPool2D + Flatten	2×2 → vector	Prepara para clasificador lineal
Linear + ReLU + Dropout	512 neuronas, p=0.5	Clasificador denso con regularización
Linear (salida)	3 neuronas + Softmax	Probabilidad por categoría

Entrenamiento y datos

- **Dataset:** imágenes de productos de e-commerce de Kaggle, categorías: aceptable, deficiente, profesional.
- **Total:** 6.000 imágenes etiquetadas · Split: 4.800 entrenamiento (80 %) / 1.200 validación (20 %).
- **Data augmentation:** flip horizontal, rotación $\pm 10^\circ$, variación de brillo.

- **Hiperparámetros:** Adam lr=0.001, CrossEntropyLoss, batch=32, 10 épocas.
- **Exportación:** mejor checkpoint (.pt) convertido a ONNX con exportModel.py para inferencia en C++.

Evaluación y métricas

Resultados reales del entrenamiento (10 épocas):

Época	Loss	Train Acc	Val Acc	Guardado
1	0.2424	95.83 %	99.17 %	✓
2	0.0578	99.21 %	99.25 %	✓
3	0.0422	99.17 %	99.17 %	—
4	0.0312	99.48 %	99.08 %	—
5	0.0274	99.40 %	99.25 %	—
6	0.0233	99.52 %	99.17 %	—
7	0.0236	99.46 %	99.17 %	—
8	0.0191	99.60 %	99.50 %	✓ Mejor modelo
9	0.0198	99.54 %	99.42 %	—
10	0.0178	99.56 %	99.25 %	—

Mejor resultado: Época 8 · Val Acc = 99.50 % · Train Acc = 99.60 % · Loss = 0.0191

Análisis crítico

- ✓ **Funciona bien:** Fotos profesionales con fondo blanco y buena luz: CNN y filtros coinciden consistentemente.
 - ✓ **Funciona bien:** Fotos claramente deficientes (oscuras, borrosas): pipeline las penaliza correctamente.
 - **Falla ocasionalmente:** Imágenes nocturnas con ruido: Laplaciano confunde ruido con nitidez. Mitigado con penalización por brillo.
 - **Falla ocasionalmente:** Fondos naturales (plantas, madera): penalizados como 'no uniformes' aunque sean válidos estéticamente.
- **Mejoras futuras:** Segmentación de fondo (GrabCut/SAM), reentrenamiento con imágenes del dispositivo real, ajuste de umbrales por categoría de producto.